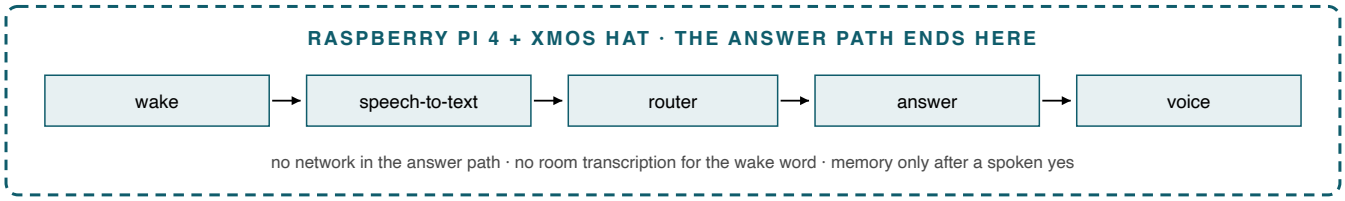
FOR OPENHOME · DEVELOPER GRANT CONVERSATION

Astral / OpenBrain
a local-first voice device on the OpenHome DevKit

The whole loop runs on the Pi 4 with the XMOS HAT: wake phrase, speech-to-text, the answer, and the voice. Nothing leaves the device in the answer path. Every number in this document was measured on that hardware or on the development Mac, and each is recorded with its date in the project's bug ledger.



PREPARED BY
Jamesmykil Weber

DATE
4 September 2026

PURPOSE
Developer grant conversation

SECTION 1

1. What is on the device today

One page. What runs, how fast, and what has been held.

The whole loop is on the Pi 4 with the XMOS HAT, and nothing leaves it. The wake phrase is a Vosk grammar with two phrases and an everything-else bucket, so I never transcribe the room, especially not while it is idle. Speech-to-text is whisper base.en, q5_1. The voice is piper. Above those sits a compiled answer kernel, `astral_kernel` 2.2.0, built aarch64 on the device; a Julia mathematics kernel running as a service; and a small model, Llama 3.2 1B Instruct at Q4_K_M, served by llama-server on port 8791. That is the whole stack. There is no network in the answer path.

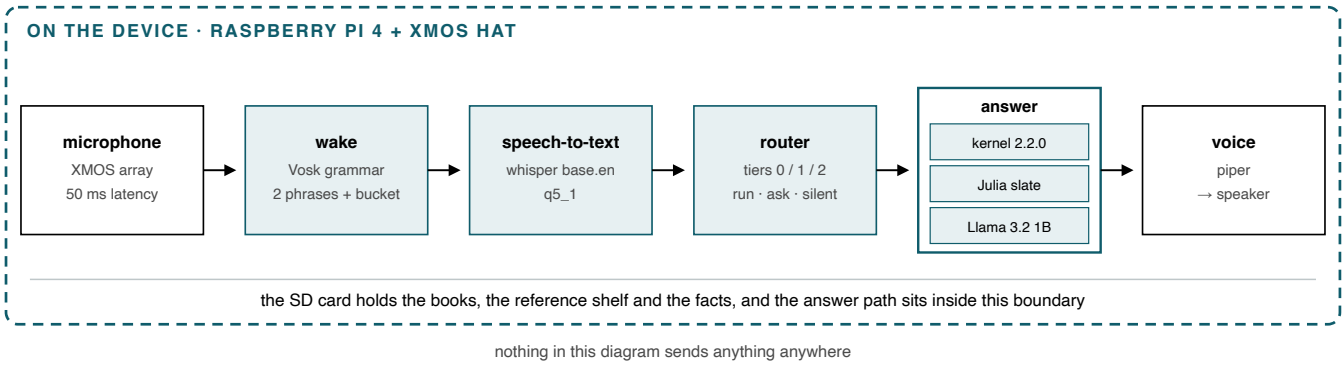


Figure 1. The loop end to end. Every component named here runs on the board; the dashed boundary is not crossed while a question is being answered.

The three levels, measured

The mechanical class covers time, date, arithmetic, unit conversion and "what can you do". It answers in one to eleven milliseconds, median, on the device. The library class, passages out of the books and encyclopedias and the curated facts, is usually sub-second warm. Comprehension in its own words is seconds. One honest number: after a cold reboot, the first library question took forty seconds once. That is the index warming off the card, and a warm-up at boot is the fix.

3,636	3,630	12 of 12	682
checks held on the DevKit, 0 failed, across 26 suites	checks held on the Mac, 0 failed	scripted voice demo lines asked and answered, 216 to 272 s	curated facts on the card, audited by hand

On the card: three books; two documentation sets across sixty-six files; a reference shelf holding Britannica, world history and physical science, about three hundred thousand passages; six hundred and eighty-two curated facts; and twenty-eight local abilities.

Under the service's own interpreter the device voice suite holds 141 checks, 0 failed. I ran the scripted voice demo under scrutiny the day before this meeting: twelve questions asked and answered by voice, twelve of twelve as scripted, and it left the settings, the hooks and the notes as it found them.

> **The discipline behind the counts.** Every new gate is proved red before it is trusted green. A check that has never failed is not known to check anything.

SECTION 2

2. The levels it can hit

No tier runs anywhere until its cost has been measured on that machine.

A script runs the engine over the corpus on the host it sits on and writes a cost profile. A second file compares that profile to a budget and produces a fits table. The router reads the table before it runs anything: a class that fits runs, a class that fits somewhere else says what it would need and asks, and a class that fits nowhere stays silent so the agent takes the turn.

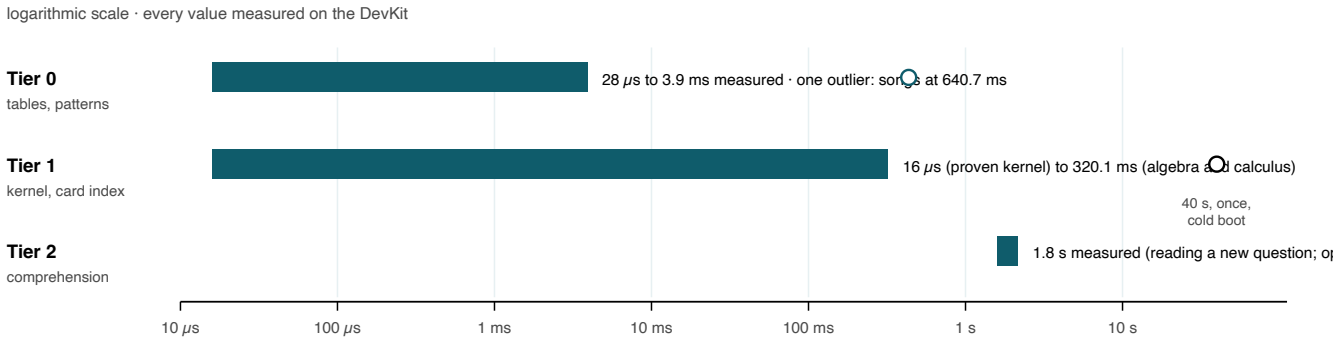


Figure 2. The three tiers as latency bands. Hollow markers are the two honest outliers: songs played from the card, and the single cold-boot library question.

Measured on the DevKit (Pi 4, 8 GB, idle, 30 runs per phrase). The figure in brackets says how it was arrived at. I take a percentile where a class is cheap enough to sample hundreds of times, and the observed maximum where it is not, because a percentile over six samples is a maximum wearing a percentile's name.

WHAT IT ANSWERS	COST ON THE DEVKIT
TIER 0 · PATTERN AND TABLE CODE	
time and date	100 µs (p95/200)
the device and its OS	261 µs (p95/40)
grades	356 µs (p95/320)
sun signs and the moon	544 µs (p95/40)
physics and the planets	793 µs (p95/1080)
statistics	808 µs (p95/480)
number tools	808 µs (p95/1000)
deciding it has no answer	870 µs (p95/1480)
chemistry	1.0 ms (p95/680)
flashcards from the card	1.0 ms (p95/40)
math, money, conversions	1.3 ms (p95/2000)
timers, alarms, reminders	2.9 ms (p95/40)
what it can and cannot do	3.2 ms (p95/40)
small talk and jokes	3.9 ms (p95/40)
songs, played and spoken	640.7 ms (p95/40)

WHAT IT ANSWERS	COST ON THE DEVKIT
TIER 1 · KERNEL OR CARD INDEX	
exact arithmetic, proven kernel	16 µs
passages from the books	1.1 ms (p95/42)
every sense of a word	26.3 ms (p95/42)
algebra and calculus	320.1 ms (p95/40)
TIER 2 · COMPREHENSION, IN ITS OWN WORDS	
reading a new question	1.8 s (max/6)
open conversation	1.8 s (max/6)

The card is what makes the upper tiers possible here rather than somewhere else: the dictionary is Open English WordNet with 135,969 lexemes and 107,519 defined senses; the comprehension core reads against 133,218 dictionary entries and 52,571 phrases from a 456 MB data set; 397,706 indexed passages sit beside them. A LAN route exists for anything the device cannot hold, and today nothing in this table needs it.

SECTION 3

3. Mechanics of the code

Wake, burst-record, transcribe, route, speak. The loop runs half-duplex, because this HAT has no hardware echo cancellation.

After every sound the device makes, it flushes the microphone and holds a refractory window, so it cannot answer its own voice. Interruption sits on top of that as an echo-gated check rather than an open microphone.

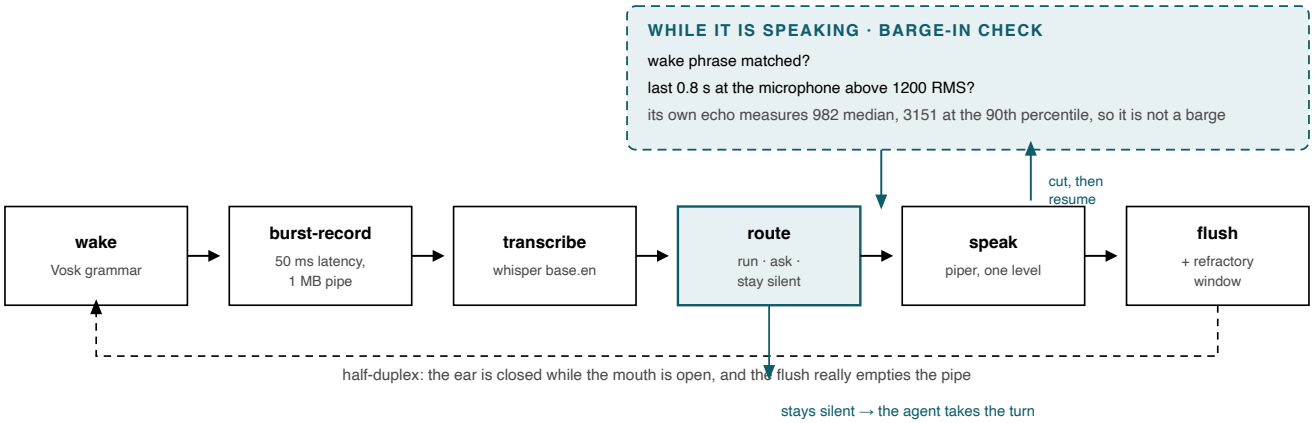


Figure 3. One turn. The barge-in check is a side loop off "speak", not a second listener: a false barge is harmless, because the sentence resumes from the point before the cut.

The router and the tiers

One place decides run, ask, or stay silent. It names the command class a phrase needs, asks a fits table what this host may do with that class, and returns something the loop can act on without a second opinion. Tier 0 is pattern and table code, microseconds. Tier 1 is a native kernel or an indexed corpus on the card. Tier 2 is comprehension. Every class carries a measured cost profile for the machine it is on.

Budgets inform; they do not block

The budgets file holds a ceiling per tier per host. The library index grew from two hundred and thirty-one thousand passages to three hundred and nine thousand, the measured 95th percentile crossed the ceiling by two hundred and fifty-eight microseconds, and the device correctly and completely switched its own library off, and every book question answered with nothing. A ceiling that kills a capability at a margin of three hundredths of a percent is a number chosen wrongly, not a capability that failed. Storage classes carry their own budgets now, because a file write is not a table lookup. A class I measure slow goes to a second lane, "I'll look that up while we talk", with the answer landing in a pause rather than over a sentence.

The ear: two seconds, then fifty milliseconds

`parec` opened with no stated latency inherits pipewire-pulse's default record fragment, and that is two seconds, so every wake phrase, every interruption and every flush was acting on a room that no longer existed. At `--latency-msec=50`, with the pipe widened to 1 MB so a flush really empties it, the same sound is heard 0.66 s after its player is launched. With the 64 KB default, 7.9 s of stale audio arrived in the second after a flush.

Memory and consent

Nothing is written until somebody says "remember me". It is a property graph on the card, with a clock reading on every row. "Forget me" deletes the file, not the rows. Notes mode is separate and announces itself. Conversation context lives in process memory, times out, and is never written down.

SECTION 3 · CONTINUED

3. Mechanics of the code (continued)

Echo-gated barge-in, with resume

A wake matched while the device speaks counts only if the last 0.8 s at the microphone is above 1200 RMS; its own echo measures 982 median and 3151 at the 90th percentile, and produced no wake match in 18 s through a ready recogniser. A barge that comes to nothing, with nothing usable or the wrong language, finishes the sentence from the point before the cut instead of losing it. Three false wakes in a row switch barging off until a real answer. The recogniser is reset after every uninterrupted sentence, and whatever queued on the microphone while the voice was being made is dropped before it plays.

One loudness model

Measured as RMS at the device's own microphone, the sounds were never at one level.

Measured on the device as RMS of a 16-bit sample. Five to one between loudest and quietest before any setting touched them, which is why no setting could make them consistent.

SOUND	RMS AT THE MICROPHONE	AFTER THE FIX
wake chime	16,862	every chime levelled to 6,750
dismiss	13,585	every chime levelled to 6,750
accept	10,262	every chime levelled to 6,750
the tick	5,776	levelled, then the 1.5× lift
speech	~4,500	sentences at 3,900 to 4,500
the ready tune	3,146	every chime levelled to 6,750

Every file and every sentence is brought to one target level before its relative and the chime lift apply. Chimes and the tick carry a fixed 1.5× lift that speech never gets, because a short transient at the same peak as a sustained voice is heard as quieter. All of it sits under one master that only a person moves. Shipped at master 65, chimes 70, tick 70, speech 100.

The mathematics kernel

Julia, with an Ada/SPARK oracle behind a C ABI. It takes about forty-three seconds to come up on this board and milliseconds after that, so one process owns it and both callers talk to it over a Unix socket. Before that, the ability paid the start-up on every turn and then offered to send the question away, so a local-first device gave up exact mathematics it could do, because of a process boundary. Exact arithmetic is computed twice, by Python's fractions and by the SPARK oracle, and the two must agree before a word is spoken. A disagreement is a silence, never a guess.

The compiled kernel as a shim

OpenHome's contributing rules are MIT, readable and security-scanned, and a local ability is three files, one of them named `devkit_functions.py`. So the ability is a thin MIT shim, three hundred and forty-three lines: it finds the hub, falls back to the compiled kernel, and if neither is there it says why rather than going silent. The engine ships the way any Python dependency ships, as a wheel named in `requirements.txt`, which OpenHome's own local-ability documentation describes as the way to bring code onto the DevKit.

> **The seam is documented, not hidden.** Nothing proprietary hides inside the MIT file, and nothing in the MIT file needs the engine to be readable in review. `BOUNDARY.md` states where the line is; the review surface is the shim, which is the whole surface touching the platform.

Test discipline

Twenty-six suites, each named for what it proves, run against a scratch copy of the device's state so a check can never touch the card. One suite's only job is to prove that nothing fails silently: it reads the refusal reasons out of the source and fails the build if any path can go quiet.

SECTION 4

4. Expansion: the route ladder

Four rungs, with consent by name. Only the local rungs are live.

The ladder exists in code today: mechanical here, a model here, a machine in the house, then a named cloud provider. Consent is by name. The device says which machine in the house, or which provider, it would send the question to, because "the cloud" is not something a person can choose between. The cloud hand-off is designed and not connected, and that is deliberate: the optional cloud gets added when the ranking reaches a level that needs it. Today no class is in that position, so the device does not make the offer. A route that cannot answer is worse than no route.

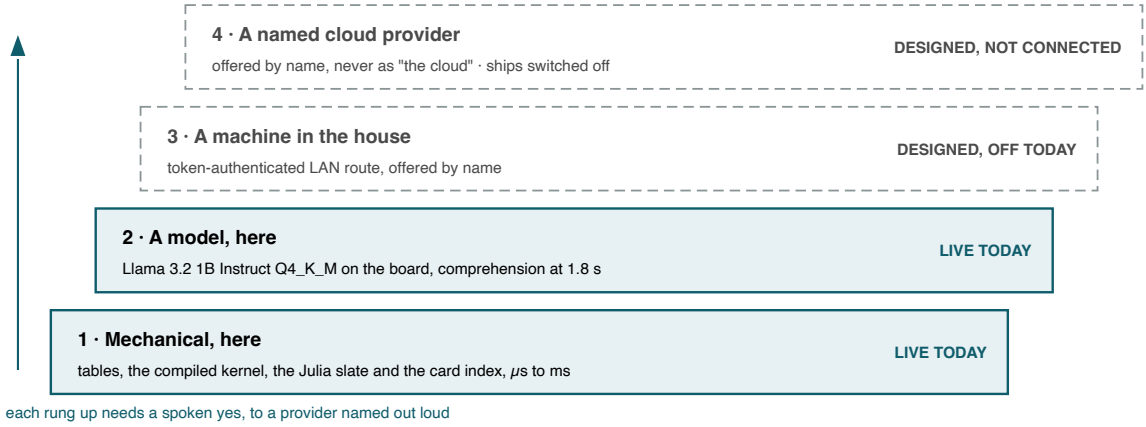


Figure 4. The route ladder. Solid rungs run today; dashed rungs exist in code and are switched off, because no measured class currently needs them.

The near list

- **A trained wake model from real recordings**, so a false-wake number in a room with a television becomes a measurement instead of an estimate.
- **Sound packs already ship.** OpenHome's house set is the default, and any folder dropped into the packs directory is a pack, chosen by voice: "use the astral sounds". The `astral` pack is real recordings with attributions carried on the card.
- **Languages.** It hears about a hundred and says which one it heard, and it answers in English, rather than translating its own sentences with a 1B model that invents names.
- **A second household**: a DevKit running a month in a home that isn't mine. That's the test I can't run alone.

> **What OpenHome gets from the ladder.** Every turn answered on the device is a turn that costs the platform nothing, and works when the network does not. The rungs above exist so a class that needs a bigger machine can still be answered, with the person's consent, given to a provider by name.

SECTION 5

5. The bridge to the bigger work

The DevKit work is the smallest node of a local-first system I run, which is why it is built the way it is.

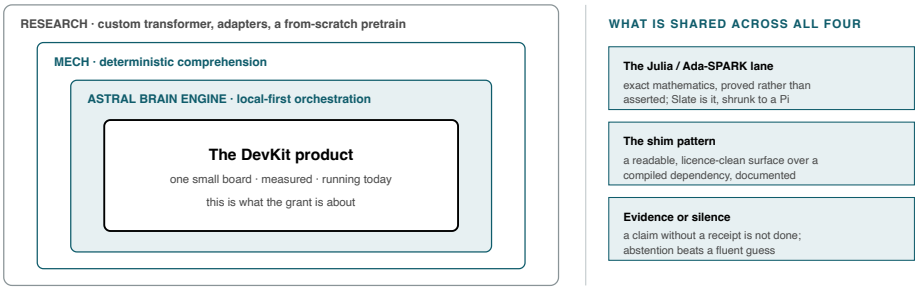


Figure 5. The device sits inside three larger layers. The right-hand column is what travels between them: the same lanes and the same seam.

I have been building this on my own for about three years, and the Astral engine has been the centre of it since spring.

The orchestration engine

It is model-agnostic orchestration with a delegation hierarchy, persistent memory, budgets and a task board, and it routed the work on this project. Prime directives set what any process may do, product isolation keeps two projects from being conflated into one build path, and a coordination bus makes a process claim the work before it touches anything.

MECH, the deterministic comprehension engine

It carries no model weights and does no sampling. Its law is that a reading becomes a *belief*, not a fact: revisable, carrying its inferential provenance and a condition that would overturn it, and only a gate promotes anything to actual. It stores a sentence one of two ways. The first reads "this document states…", with no subject bound, and by design it can never be promoted. The second is a typed reading with a real subject, predicate and object, and that one can. Measured on the live ledger, 99.9% of a seventy-gigabyte library landed in the first lane. The engine is telling the truth about what it did not understand. The remedy is a re-read with comprehension on, which typed 17.5% of one documentation set. It abstains rather than guesses.

It has the same shape the device does: a Julia lane for exact mathematics; an Ada/SPARK lane where I prove float behaviour instead of asserting it, with thirty-five open verification conditions closed by running the prover per subprogram rather than whole-unit; and Rust libraries behind one C ABI. Slate on the DevKit is that same Julia lane, shrunk to fit a Pi.

The model side

Two of my fine-tuned models already run, a role-tuned 4B in July and a behaviour-tuned 0.8B in August. A from-scratch transformer I call the subconscious is 34,000 steps into pretraining on the Mac mini, and it is the component that may only propose, never answer.

I train on a 16 GB M4 Mac mini that panics when its swap fills, a Linux box with 23 GB of RAM, an RTX 2060 with 6 GB that cannot hold a training run, and an RX 580. The training lane is the bottleneck in all of it.

None of that is finished. The point is that the DevKit product is what happens when I aim the same discipline at one small board, and there, measured, today, it works.

SECTION 5 · CONTINUED

5. Research areas, and where the information comes from

The route from this device to the larger systems is work and funding. Each row says what runs today, what it stands on, and what comes next.

AREA	WHAT EXISTS TODAY	WHAT IT DRAWS ON	NEXT
The mechanical assistant this device	3,636 checks held on the DevKit across 26 suites and 3,630 on the Mac, 0 failed. Tier 0 answers between 28 µs and 3.9 ms. Twelve of twelve demo lines answered by voice.	The library and the 682 facts on the card, and the open-source speech stack.	Milestones 1 to 4.
Local inference	Two lanes I run: a Rust server with candle serving GGUF on the CPU, and an MLX lane on the GPU running my own server rather than the stock one. Two fine-tuned models: a role-tuned 4B on 1 July, LoRA r8 over 300 iterations, and a behaviour-tuned 0.8B on 28 August. The from-scratch transformer I call the subconscious sits at step 34,300, validation 2.31.	Open weights fetched once onto my own drives, my own training data, and the MLX and candle runtimes.	The comprehension tier trained and evaluated on the machine.
MECH comprehension	A reading becomes a belief that stays revisable, carries its inferential provenance and a condition that would overturn it, and only a promotion check can make it actual. Two reading lanes, measured on the live ledger: 120,927 claims at 99.9% in the bridge lane with no subject bound, and 136 typed readings. It abstains rather than guess.	The books I own and attested, a 605 MB ledger indexed into one 727 MB SQLite file, and the toddler baseline of 3 books and 22 sentences.	A re-read with comprehension on, which typed 17.5% of one documentation set, then the reading lane on the DevKit.
Verified mathematics	Julia computes the exact answer and Python re-derives it, so the pair decides instead of approximating. In Ada/SPARK I closed 35 open verification conditions by running the prover per subprogram at level 2, with level 3 for float multiplication monotonicity. Rust libraries sit behind one C ABI.	Julia, Ada/SPARK with gnatprove, and Rust.	It proves what it can and refuses the rest: a codec body outside SPARK is labelled assumed and never proved, and a block whose scale saturates is rejected rather than silently zeroed.
The Astral Brain Engine	Local-first orchestration under twelve prime directives. PD-1, evidence or silence, refuses a claim that arrives without a receipt. PD-3, nothing orphaned and nothing conflated, resolves every file I touch to one product and no other. PD-4, plan then prove then build, holds destructive work until I authorise it by hand. A coordination bus makes a process claim the work before it touches anything.	The product map, the receipt ledger and the bus, all on my own machines.	Move each directive from advisory text into code that enforces it.
The knowledge shelf	On the card: 57 reference files and a 660 MB index over 397,706 passages, holding the 24 Encyclopedia of Physical Science and Technology volumes, Britannica across 29 files, the seven-volume World History set, the Encyclopedia of Computer Science, and Ancient and Forbidden Knowledge. Staged and converting today: 65 DK volumes, 6.4 GB of PDFs.	Encyclopedias and books I bought and own, converted to text on my own hardware.	Build the index on a real machine and ship it built, so the card gets the full shelf without hours of indexing on a Pi.

Sources

The open-source stack: whisper.cpp, piper, vosk, llama.cpp, openWakeWord, PipeWire, SQLite FTS5, Julia, Ada/SPARK and Rust. The encyclopedias and books I own. My own corpora and training runs. The measurements ledger in the repository, KNOWN-BUGS.md, where every number carries its date. OpenHome's SDK and its local-ability contract. My coursework at SNHU, where I am reading for a BS in Computer Science with a Software Engineering concentration.

> Every row above is running or measured today. The machine and the time are what turn the next column into the exists column.

SECTION 5 · CONTINUED

5. Previous work

The work I'm proudest of is the harness and the engines under it. This device is the small end of that, and the first piece to ship through someone else's review.

PROJECT	LANGUAGE	AVAILABILITY	WHAT IT IS
Astral Brain Engine	Python, Rust, Shell, Dart	private	The harness. A model-agnostic orchestration layer with a delegation hierarchy, persistent memory, budgets and a task board. 607 commits. It routed the work on this project.
Strata (Astral MECH)	Julia, Rust, Ada SPARK	private	A language comprehension engine with no model weights and no sampling: a typed role graph for every sentence, memory that keeps its sources, byte-identical output for the same input, and an honest refusal when it cannot read. 436 commits. Ships as a signed macOS app over a Flutter surface.
Slate	Julia	private	The mathematics lane. It runs on the DevKit today as the kernel service.
Q OS (QNX, QOS)	C++, Rust	public	A windowed desktop environment for the Nintendo Switch that replaces the home menu, with a real file manager. Verified on hardware: Erista, Atmosphere and Hekate. `https://github.com/Jmesmykil/QNX`
Void IDE	Swift	private	A native macOS editor that speaks to the engine. 82 commits.
Void CLI	Rust	private	A terminal for the engine, with six or more model providers behind one binary. 86 commits.
Astral Workspace, BrainManager, AstralEngineKit	Swift	private	The macOS apps and the shared package around the engine daemon.
ASCII	JavaScript	private	A glyph-grid render kernel that composites any signal on the GPU in one draw call. 77 commits. It hosts ASCII Studio for Unity 6, a shipped package, ASCII Studio for Blender, and Tessera: 3D models in, sprite sheets out, with no image model in the path.
sCAN (CryptoBuddy)	HTML, JavaScript, Rust, PostgreSQL	private	A desktop-style crypto research workspace with a Rust gateway. 179 commits.
SlimeShot	C#, Unity	private	A game with its own physics kernel. Close to done.
MoBA	C#, Unity	private	A server-authoritative online game, close to mechanically done. 579 commits.
soloDOLO	Python, Blender	private	An animated series in production. 87 commits.
SourceBound	C#	private	A 2D adventure RPG that teaches coding by having the player code regions into existence.
Filament	Swift	private	A hardware-in-the-loop workbench for firmware work with local models.
EMULive	Python, Rust	private	A living Game Boy emulator with a mind layer. Alpha.
astral-OpenBrain	Python	public	This project. `https://github.com/Jmesmykil/astral-OpenBrain`
openhome-cli	TypeScript	public	The CLI that deploys and assigns OpenHome abilities from the terminal. `https://github.com/Jmesmykil/openhome-cli`
abilities	Python	public	Open-source abilities for OpenHome agents. `https://github.com/Jmesmykil/abilities`

> **The pattern across all of it.** I build local first, with no cloud in the path unless I choose to put it there, and I ship tools other people use. Counts above are commit counts, and repositories I forked from other authors are left out on purpose.

SECTION 6

6. The ask: five milestones, and what each level buys

The full grant is $50,000. Below is what each level buys, down to nothing, because I'm going to keep building this either way.

Every hour of compute today comes out of my own pocket and every test runs on one Pi. The advancement is a fully mechanical assistant: a device that answers most of a household's questions on its own hardware in milliseconds, with no model call and nothing sent anywhere, and says so when it can't. That runs today. Each level buys more of it.

Figure 6. The five milestones.

What each milestone delivers, and what it costs.

MILESTONE	WHAT IS DELIVERED	COST
1. The mechanical assistant, finished	Every exact-answer class complete: time, date, math, money, units, definitions, facts, the library, timers, notes and memory. The no-silent-failure suite held at zero failures, the kernel wheel published, the ability installable from the OpenHome catalogue, the runbook and the bug ledger current.	my time
2. The ladder, live	A machine in the house as the LAN rung and named cloud providers as the last rung, each offered by name and consented to by voice, measured end to end.	the machine, provider credits, my time
3. The library at full size	The encyclopedias I already own, with 65 DK volumes on the card and indexed today, on the card. The index gets built on a real machine and ships built, so an offline device answers from an encyclopedia in under a second.	the machine, my time
4. Measured in other homes	Two more DevKits in homes that are not mine, a month each, with false wakes, loudness and every failure read out of the logs and fixed.	two DevKits, speakers and microphones for testing, travel, my time
5. The comprehension tier	My own inference and transformer work applied to the one tier that still uses a stock model, trained and evaluated completely on the machine, so the device can hold a conversation the way it holds a fact.	training compute, frontier model access for evaluation, my time

Where I'm building from today

I built everything here on hardware I already owned and paid for myself: a 16 GB M4 Mac mini that panics when its swap fills, a second Linux box with 23 GB of RAM, an RTX 2060 with 6 GB that cannot hold a training run, an RX 580, and the DevKit. Over the last one to three years I have spent about $10,000 on this work in total, all out of pocket, most of it renting compute I would rather own. Two fine-tuned models of mine already run, a role-tuned 4B in July and a behaviour-tuned 0.8B in August, and a from-scratch transformer I call the subconscious is 34,000 steps into pretraining on the Mac mini. I'm a student with no income. That is the ceiling the machine removes.

Hardware, by level

I'm upfront about my limits because they are the reason I'm asking. Today the training lane is a 16 GB Mac mini and a 6 GB RTX 2060 that cannot hold a training run. There are levels, and each one buys real speed.

- **$2,000 to $5,000.** A GPU with real memory, plus RAM for my second machine. That alone moves training and the library index off the mini and onto a card that can hold them.
- **$3,800 to $4,750.** A Minisforum MS-S1 MAX with 128 GB of unified memory, a Ryzen AI Max+ 395, Radeon 8060S graphics and a 2 TB SSD, at $3,799 on the maker's store today, or a Mac Studio at the same level. 128 GB the GPU can use holds a 70-billion-parameter model quantised.
- **The full build.** A board with two full-lane PCIe slots, two cards at 32 to 48 GB of VRAM each, and the RAM to match. That is the machine that trains the comprehension tier and serves the LAN rung at once.

How long this has taken. About three years of research on my own, the Astral engine since spring, and the OpenHome work over the last month.

SECTION 6 · CONTINUED

6. The ask (continued): what each level buys

Every level below is real. The bottom one is what happens if nothing comes of this conversation, and I have written it down so nobody has to guess.

Levels top to bottom. The workstation is the line that changes the most for the least.

LEVEL	WHAT IT BUYS
$50,000	All five milestones. The machine, the hardware for testing, the training compute and frontier model access for milestone 5, and part-time help with packaging and review so milestone 1 lands in weeks instead of months.
$10,000	Milestones 1, 2 and 3 in full, and the machine: a fully mechanical assistant, installable, with the ladder live and the library at full size.
$3,800 to $4,750	The machine, and with it milestones 1 and 3.
$500 to $2,000	One more DevKit and test hardware. Milestone 1 on my own time.
Nothing	I keep building it on the Pi, out of pocket and slower, with no second device and no machine.

What OpenHome gets

- Every turn answered on the device costs the platform nothing, and works when the network does not.
- A supported local path on the DevKit, rather than one developer's card.
- A household appliance that answers from an encyclopedia with no network at all.
- A developer who ships. Three public repositories in your ecosystem are already mine.

The three things I want out of this conversation

- **One.** Which conversation this is: grant, sponsored integration, or both.
- **Two.** A paid, milestone-based integration, so this is a supported path on the DevKit rather than one developer's card.
- **Three.** Trigger words registered on agent 595324, and whether the wake phrase should stay "open home".

Who is on it

Me. At the full amount I'd bring in part-time help for packaging and review so milestone 1 lands in weeks.

SECTION 7

7. The demo, line by line

Twelve questions, asked and answered by voice, with nothing sent anywhere. Every step below is nameable in the code.

The pipeline is the same for every one of the twelve lines. What changes between them is which handler the router reaches, and how long that handler takes.

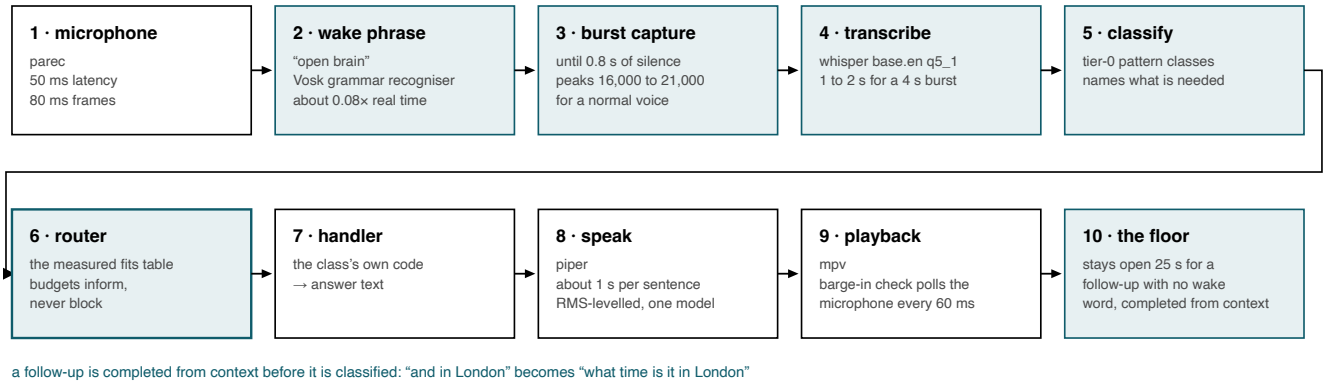


Figure 8. One turn, with the parameter that governs each stage. Stages 2, 3, 5, 6 and 10 are the ones that changed most during the week before this meeting.

The twelve scripted lines in the order they are asked, with the class the router names, the tier it resolves to, where the answer physically comes from, and the time measured on the device.

#	THE QUESTION	CLASS · TIER	WHERE THE ANSWER COMES FROM	MEASURED
1	"how many books do you have"	library · tier 1	the SQLite index on the card (660 MB)	878 ms warm 40,033 ms cold, once
2	"what page are lambdas on"	library · tier 1	the back-of-book index	81 ms
3	"what do the books say about entropy"	library · tier 1	full-text search across the books, then the reference shelf (~400k passages)	sub-second warm
4	"tell me something interesting"	facts · tier 0/1	the 682 curated facts on the card	milliseconds
5	"what time is it in tokyo"	mechanical · tier 0	the device’s own clock and time zones	~1 ms
6	"how many kilometres in twelve miles"	calc · tier 0	exact arithmetic	~1 ms
7	"what is one third plus one sixth"	calc · tier 0	exact fractions, checked against the Julia slate kernel where it runs	~1 ms
8	"tell me a riddle"	facts · tier 0	the curated facts on the card	milliseconds
9	"open the pod bay doors"	small talk / meta · tier 0	the tier-0 small-talk table	milliseconds
10	"what is the capital of Mongolia"	comprehension · tier 2	the local model, Llama 3.2 1B Q4_K_M via llama-server, and it abstains	2,267 ms
11	"why didn’t that work"	meta · tier 0	its own recorded reason for the abstention	~1 ms
12	"what can you do"	meta · tier 0	the ability registry	3 ms

SECTION 7 · CONTINUED

7. The demo, line by line (continued)

	THE LINE, AND WHERE ITS ANSWER COMES FROM	TIER	MEASURED
1	**“how many books do you have”** library index on the card, SQLite, 660 MB	tier 1	**878 ms**
2	**“what page are lambdas on”** the back-of-book index	tier 1	**81 ms**
3	**“what do the books say about entropy”** books, then the reference shelf, ~400k passages	tier 1	**sub-second**
4	**“tell me something interesting”** 682 curated facts on the card	tier 0/1	**ms**
5	**“what time is it in tokyo”** the device clock and its time zones	tier 0	**~1 ms**
6	**“how many kilometres in twelve miles”** exact arithmetic	tier 0	**~1 ms**
7	**“what is one third plus one sixth”** exact fractions, checked against the Julia slate kernel	tier 0	**~1 ms**
8	**“tell me a riddle”** the curated facts on the card	tier 0	**ms**
9	**“open the pod bay doors”** the tier-0 small-talk table	tier 0	**ms**
10	**“what is the capital of Mongolia”** the local model, and it abstains rather than guess	tier 2	**2,267 ms**
11	**“why didn’t that work”** its own recorded reason for the abstention	tier 0	**~1 ms**
12	**“what can you do”** the ability registry, 28 abilities	tier 0	**3 ms**

line 1 was 40,033 ms once, the first time after a cold reboot, while the index was read off the card

Figure 9. The run in order. Ten of the twelve resolve in tier 0 or tier 1, on the card or in the compiled tables; one goes to the local model, and that one abstains.

The two lines that matter most

Line 10, “what is the capital of Mongolia”. It goes to the local model and comes back with nothing it is sure of, so it says “I couldn’t work that one out, and I won’t guess.” That is deliberate: a reading may form a belief, not a fact, and a confident wrong answer is worse than a refusal.

Line 11, “why didn’t that work”. A tier-0 meta class reads the recorded reason back: “It went to the part of me that works things out, and that came back with nothing it was sure of. I would rather say nothing than invent an answer.”

The run, measured

Lines asked, and answered as scripted	12 of 12
Not as scripted	0
Median line	8 to 11 ms
Slowest line, warm (line 10)	2,267 ms
Slowest line, cold (line 1)	40,033 ms
Whole run, warm	216 s
Whole run, after a cold reboot	272 s
Settings, hooks and notes after the run	unchanged

What to watch during the live run. The chimes mark the stages without words. There is a cue on wake, and a working tick while speech-to-text runs. The floor stays open for twenty-five seconds afterwards, so a follow-up needs no wake word.

SECTION 8

8. Proof and honesty

Figure 7. The line between the two columns is the point of this page: nothing on the right is claimed as working, and nothing on the left is claimed without a dated measurement.

The known limits, stated plainly

- **Barge-in needs the person to be louder than the device.** With an interrupter played from the device's own speaker at equal loudness, the recogniser caught the wake phrase in the controlled trial, 1.2 s after the phrase began, but not inside the loop, where speech plays at the trim gain.
- **Echo cancellation is available but not usable yet.** PipeWire's `module-echo-cancel` with WebRTC is installed on the DevKit; loaded at runtime it creates `ec_source` and `ec_sink`, but under the pro-audio profile the source delivered zeros. The graph needs `pw-link` work before it is usable. I have not shipped it, and the path is properly mapped.
- **The microphone came up at the wrong level after this morning's reboot.** Our unit and the OpenHome node server both set a default at boot, and whoever ran last won. The loop pins the microphone and the speaker from the settings now, and holds both properly while it runs.
- **The ear can come up stuck after a boot.** When the capture stream delivers nothing, the loop kicks its own stream rather than sitting there deaf.
- **The first library question after a cold boot was slow once, at forty seconds** (40,033 ms). That is the index being read off the card for the first time; the fix is a warm-up at boot.
- **False wakes are observed, not yet rated.** With a video playing near the device it woke a handful of times, each dismissed with the tone; the chime goes quiet after three in a row. I will not quote a rate until I train a model on real recordings. Other limits are in the ledger, dated: the phone rung being off, and OCR damage in the scanned reference material, which it reads as it stands.

SECTION 9

9. Contact and next steps

Contact

Jamesmykil Weber
Sole developer: architecture, kernel, router, library, audio, deploy, tests

mykiljames253@gmail.com
+1 253 392 9204

`https://github.com/Jmesmykil`

I'm in the OpenHome Discord and in touch with your developer relations team.

Next steps

- Decide which conversation this is: grant, sponsored integration, or both.
- Agree the award level and the milestones it buys.
- Register the trigger words on agent 595324, and agree where the kernel wheel lives, PyPI or a release asset.
- A second call once milestone 1 is installable from the catalogue.

Reproducing it: one deploy script, and the demo runs by voice, "open brain, run the demo".